

Module 33: SOLID Principles & Clean Design

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

With the full set of C++ OO features in hand, this module turns to organising code so it stays easy to change. Students learn coupling and cohesion, then the five SOLID principles - each through a before/after example in one file - and connect them to work they have already done: tell-don't-ask, composition, interfaces, polymorphism. A catalogue of code smells gives them words for what feels wrong in code, and a disciplined refactoring method (save the output, take small steps, commit each one) lets them improve structure without changing behaviour. The project is a genuine refactoring of a deliberately smelly god class.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain coupling and cohesion, and aim for high cohesion and low coupling.
- Apply the Single Responsibility Principle by splitting classes with several reasons to change.
- Apply the Open/Closed Principle by replacing type switches with polymorphism.
- Recognise Liskov Substitution violations and fix the hierarchy.
- Split fat interfaces according to the Interface Segregation Principle.
- Apply the Dependency Inversion Principle with interfaces and dependency injection.
- Name common code smells and refactor safely in small, verified, committed steps.

Prerequisites

- Module 29 (interfaces) and Module 27 (composition).
- Module 22 (multi-file projects) - the project produces many new files.

Materials & Instructor Prep

- 33-solid-principles/examples/ 01 to 05, each with before and after namespaces.
- A printed or projected code-smell table from the README.

Key Vocabulary

Term	Definition
Cohesion	How closely the contents of one class belong together; aim high.

Term	Definition
Coupling	How much one class depends on the details of another; aim low.
Single Responsibility (SRP)	A class should have one reason to change.
Open/Closed (OCP)	Code should be open for extension but closed for modification.
Liskov Substitution (LSP)	A derived class must work correctly anywhere its base class is expected.
Interface Segregation (ISP)	No class should be forced to depend on methods it doesn't use.
Dependency Inversion (DIP)	High-level code and low-level details should both depend on abstractions.
Dependency injection	Passing a class the objects it depends on (usually through its constructor) instead of letting it create them.

Lesson Outline & Instructor Talking Points

1. The cost of change; coupling and cohesion (10 min)

- Ask students about the last time a small change to their code broke something unrelated. Use their answers to define coupling.
- Define cohesion with the FeeAccount vs 'SMS and PDF and fees' example, and state the goal: high cohesion, low coupling.

2. SRP and OCP (20 min)

- Run examples/01_srp.cpp. Have students name the three groups of people who might ask for changes to the 'before' ReportCard.
- Run examples/02_ocp.cpp. Point out that SolarWaterHeater was added without editing a line of existing code.

Instructor note: Warn against over-applying SRP: 'one reason to change' is not 'one method per class'.

3. LSP and ISP (20 min)

- Before running examples/03_lsp.cpp, ask whether a Square is a Rectangle. Then show resizeBillboard() failing for a Square, and the fixed hierarchy.
- Run examples/04_isp.cpp: the bulb forced to implement readTemperature() vs small interfaces that also catch misuse at compile time.

4. DIP and dependency injection (15 min)

- Run examples/05_dip.cpp. The same FeeReminderService works with SMS, email and a fake - with no edits.
- Preview Module 35: the fake sender is how we will test code without sending real messages.

Instructor note: DIP is the principle with the biggest practical payoff in the rest of Part 3 - it is what makes testing (Module 35) and patterns (Module 34) work.

5. Smells and safe refactoring (25 min)

- Walk through the smell table and have students find three smells in the exercise's CanteenManager.
- Demonstrate the refactoring loop on one small step: save output, extract a class, rebuild, diff, commit.

Common Student Misconceptions

Misconception	How to Address It
"SRP means each class should have only one method."	It means one reason to change. A class with many methods about one responsibility is cohesive and fine.
"If it's an 'is-a' in the real world, inheritance is correct."	Inheritance must be correct for the operations in the program. A Square breaks Rectangle's setter promises, violating LSP.
"More interfaces are always better."	An interface with one implementation and no prospect of another (or of a test fake) adds reading cost for no benefit.
"Refactoring and adding the new feature at the same time saves time."	When something breaks you can't tell which change caused it. Refactor first, commit, then add the feature.

Exercises & Module Project

Exercise: 33-solid-principles/exercises/exercise1.cpp - refactor a CanteenManager god class (menu, pricing, payment-type switch, receipts, hard-wired notifier) into SOLID classes, proving identical output with diff, then add a Voucher payment method without editing existing classes. Project (three tracks): Generic - SchoolManager; EE - EnergyController; Biomedical - WardManager. Each is a smelly god class to be refactored with identical output, committed step by step, and then extended with new features mostly by adding classes.

Facilitation guidance: Insist on the safety net: before.txt and after.txt must be identical after every refactoring commit. Read the commit history - many small, well-named commits are themselves evidence of the skill. In Step 4, count how many existing files each student had to edit for the new features; fewer, well-justified edits show a better design.

Discussion & Comprehension Check Questions

1. Name two smells in a class that switches on a type string to decide behaviour.

Guidance: Switch on type (breaking OCP) and primitive obsession (the type as a bare string); often also a long method.

2. Why does FeeReminderService taking a MessageSender& make it easier to test?

Guidance: A test can pass a fake sender that records messages, so the reminder logic is checked without sending anything real.

3. How can you prove a refactoring didn't change behaviour?

Guidance: Save the output (or run tests) before, make one small change, and compare the output (or rerun tests) after every step.

Assessment Ideas

- Give a 60-line class and ask students to list its smells and the principle each breaks.
- Students propose (on paper) a SOLID split of a god class, with a UML sketch.

Answer Key Notes

See [33-solid-principles/exercises/solutions/exercise1_solution.cpp](#) - its output is identical to the original's.

Wrap-Up & Bridge to Next Module

Bridge forward: "SOLID tells you what a good design looks like. Next module gives you a catalogue of proven designs to reach for - the classic design patterns."

Instructor Tips

- If time is short, protect DIP (step 4) and the refactoring loop (step 5).
- Present SOLID as questions to ask, not rules to obey - the README's closing guidance is worth reading aloud.